

Sheet Music Follower

Katie Baakkonen
Harvey Mudd College
Claremont, CA USA
kabaakkonen@hmc.edu

Matthew Molinar
Harvey Mudd College
Claremont, CA USA
mmolinar@hmc.edu

Abstract—Aligning two data sequences in different modalities often introduces many interesting challenges. In the context of music, this can mean aligning a MusicXML file with a MIDI file. Although they describe the same type of information and have a lot of overlap, they capture different aspects of music, making the alignment problem complex. We aim to address this problem by using Dynamic Time Warping (DTW) to infer measure boundary time stamps in MIDI files based on their corresponding MusicXML files. Our modified DTW approach with Jaccard distance metrics offers improved inferences of measure boundaries in MIDI files. This is important for a variety of musical applications, such as following along in the sheet music while listening to an audio recording of a performance audio.

Index Terms—dynamic time warping, MIDI file, MusicXML file, measure boundary, Schumann

I. INTRODUCTION

This paper explores an issue that has arisen in a current research project in which we would like to construct a dataset to train a system to generate more readable sheet music. This is a task that could be accomplished by separating the music into chunks by measures. This would allow a system to follow along in sheet music measure-by-measure as it tracks the performance of a given piece. Readable sheet music is important for musicians because it makes practicing more efficient.

A significant challenge in aligning MusicXML and MIDI files lies in their different representations of music. More specifically, MusicXML files contain information that MIDI files don't and vice versa.

Therefore, in this work, we explore the alignment of different music representations in the form of MusicXML data and MIDI data. The MusicXML data contained information about the measure boundaries whereas the MIDI data did not. To predict the measure boundaries for the MIDI data, we performed DTW on the XML and MIDI files and then projected the measure boundaries from the XML file to the matching frames of the MIDI file.

More specifically, DTW was used to align the measure level chunks with the overall MIDI sequence. Evaluations of the DTW algorithm involve auditory and visual evaluations to gain some qualitative insights, and quantitative measures to evaluate similarity.

The goal was to refine the DTW algorithm to handle the challenges that arise when trying to align MusicXML and MIDI files. This work can potentially lead to novel adaptations

of the DTW that improve alignment accuracy for this specific musical application.

II. SYSTEM DESIGN

A. Data Preparation

In order to perform DTW on the score and performance data, we had to convert both files to a matrix representation of the piece. We chose to do this in a piano roll-like binary format using an $88 \times N$ matrix, where 88 was the number of keys on a piano (we were working exclusively with piano scores) and N was the number of time frames in the piece. The MIDI files varied in similarity to their corresponding XML file depending on how closely the performer played the piece to the score, with some being almost identical and some being drastically different.

1) *MusicXML Data*: The MusicXML file is a text representation of the music score. An example music score is shown in Fig. 1. We used the music21 Python library [1] to parse the XML file and create a list containing the pitch, onset, and duration for all of the notes in each measure of the piece. From there, we made a binary matrix containing all of the notes played at each time frame. The vertical axis represented the pitch of the note being played and the horizontal axis represented the time at which it was played. The XML files did not contain any absolute time data; the duration and onset of the notes were given relative to the length of a quarter note. Thus, to display the matrix in a pseudo-time format, we decided to use a frame rate of 16 frames per quarter note. For example, if a measure had 4 quarter notes there would be 64 frames in the measure, and if the piece had 20 measures there would be 1280 frames in the entire piece. The matrix containing the full piece (referred to later as the piano roll) was saved along with a list of the frames where the measure boundaries were.

We tested three different variations of the XML representation: a full representation (shown in Fig. 2), a version with



Fig. 1. Excerpt from Arabeske by Schumann.

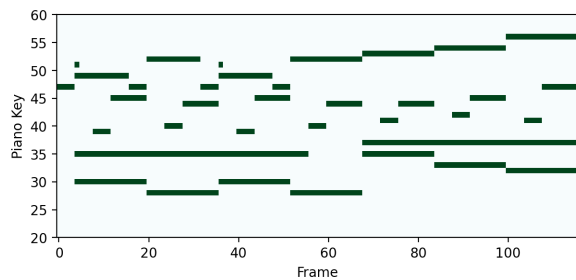


Fig. 2. Matrix representation of Schumann excerpt.

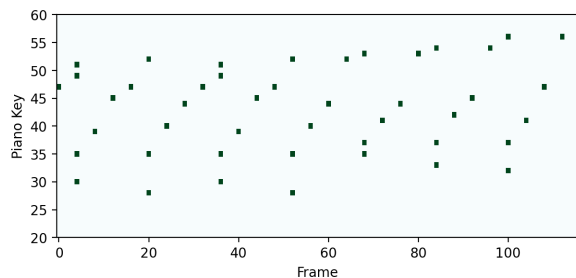


Fig. 3. Onset-only matrix representation of Schumann excerpt.

only the onsets of the notes, (shown in Fig. 3) and a version where repeated frames were excluded.

2) *MIDI Data*: The MIDI files contained data from a performance of a piece indicating the pitch that was played and at which times it was played. We used the `pretty_midi` Python library [2] to parse these files and convert them to a piano roll matrix. The MIDI files had no specified sampling frequency, so the frame rate was adjusted adaptively to allow the MIDI matrix representation to closely match the XML matrix representation in size. We made this decision to minimize the excessive time warping that could cause complications in the DTW.

We tested two variations of the MIDI representation: a full representation and an onset-only representation.

B. Standard DTW Approach

The general approach to aligning the matrix representations of a MusicXML file with a MIDI file to extract the measure boundaries from the MIDI file was to use dynamic time warping (DTW). There were two steps in this process.

First, we aligned M_{midi} with M_{xml} using DTW with binary features and a cosine distance metric. DTW finds the optimal alignment between a query sequence and a reference sequence [3]. In musical contexts, typical step size transitions of (1,1), (1,2), (2,1) with transition weights 1, 1, 2, respectively are used. This means that the tempos in the data have a maximum warping factor of two.

Then, using M_{midi} as the query, we can identify the boundaries of the matching measures from the MIDI file of the performance. Even though there was some mismatch between the MusicXML file and the MIDI file, this approach produces fairly reliable alignments, because the MIDI file is just a human interpretation of the corresponding piece. Since

the measure boundaries of M_{xml} are known, we can look at the intersection between its corresponding frame and the warping path, wp . The intersection between the two gives us the corresponding measure boundary frame for M_{midi} , which is then converted to a time stamp by dividing by the sampling rate.

C. DTW Modifications for this Application

One way in which we modified the DTW is by changing the cost metric. The cosine distance is typically used as a cost metric for a standard DTW. However, in order to take advantage of the fact that our features were binary, we used a Jaccard distance metric. Jaccard similarity is fitting because it can be described as a measure of similarity between two binary sets [4], which matches our binary feature representation. The index for this similarity ranges from zero to one, with a number closer to one indicating that the sets are more similar [5].

Furthermore, step size transitions of (0,1), (1,0), (1,1) with transition weights 1, 1, 2, respectively were used for DTW. These parameters were chosen because there were a few instances in which there were large changes in tempo. Since the standard steps for a musical context are (1,1), (1,2), (2,1), the maximum warping factor would be two. These parameters are not desirable because there are instances in which the MIDI file is faster or slower by more than two times. As a result, having only a maximum warping factor of two would degrade the alignment path because it cannot accommodate larger warping.

Finally, the process for obtaining the measure boundaries in frames from the MIDI file and converting them to time stamps remains the same as in the standard DTW approach mentioned previously.

III. RESULTS & DISCUSSION

Our experiments used the `asap` dataset [6], which contains scores and their corresponding human performances. For our evaluations, we used a diverse set of 12 pieces with varying degrees of complexity and timing variations.

Several methods were used to evaluate the performance of our system, including visual, auditory, and numerical methods. As a result, these methods allowed us to assess both the qualitative and quantitative performances of our measure boundary predictions.

A. Sonification

One approach we used to obtain a preliminary sense of the predicted measure boundary accuracy was sonification. We did this by saving our predicted measure boundaries as time stamps and using them to add audible cues. These audible cues came in the form of clicking sounds and were overlaid with the original performance. Sonification was important because it allowed us to identify subtle timing misalignments and adjust the DTW accordingly. This would not have been possible with numerical evaluations, so this step was crucial for modifying the system design.

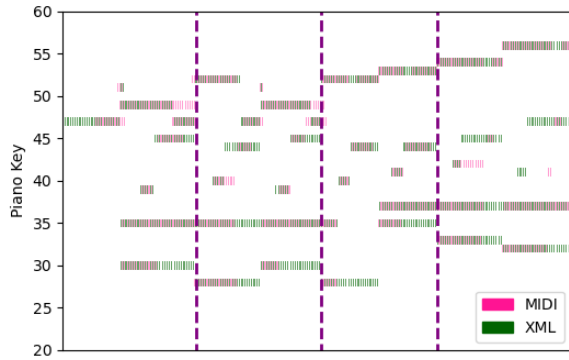


Fig. 4. MIDI and XML frames in aligned path for Schumann excerpt.

B. Ground Truth Comparisons

To establish a numerical baseline for evaluating the measure boundary inference performance, we manually annotated a set of measure boundaries for pieces by Bach and Schumann. With the manually created boundary labels, we were able to compare our DTW’s predicted measure boundary locations with ground truth measure boundary locations at various tolerance levels, providing insight on the accuracy of the inferred measure boundaries.

With the manually labeled measure boundaries, we were able to compare them to the predictions with audio, which sounded very accurate to the human ear. We were also able to visualize the alignment predictions and the manually labeled measure boundaries to get a visual sense of how accurate the predicted measure boundary locations were.

1) *Visual Evaluation:* Fig. 4 shows the side by side frames of the Schumann excerpt XML and MIDI files that the DTW predicted to be aligned. At times, mismatches can be seen in which either the MIDI file or the XML file had notes that were not being played in the other file. The purple dashed lines show where the predicted measure boundaries were. This type of visualization allowed us to compare the predicted measure boundaries with the measure boundaries that were given in the score.

Fig. 5 displays the accuracy of the measure boundary predictions at different tolerance thresholds. At stricter tolerances (below 50 ms), the accuracy remained relatively low, reflecting the timing variations present during a human performance. However, at tolerances beyond 70 ms, the accuracy improved, which makes sense given that there are bound to be timing mismatches in a human performance. This improvement in accuracy can also be explained by the fact that the sampling rate was around 20 Hz. These tolerances beyond 70 ms are more significant because a human would not be able to discern smaller millisecond differences.

C. Note Metrics

Using insights from audio analysis, ground truth measure boundaries, and visual analysis, we developed two note-based metrics: soft-similitude and strict-similitude. We used these metrics to quantitatively evaluate the effectiveness of the

modified DTW algorithm. In both cases, we introduced a value, epsilon, which controls how many frames before and after the initial measure boundary prediction we analyze.

The soft-similitude metric is calculated by comparing one MusicXML frame with the corresponding window of MIDI frames. The window of MIDI frames that is being analyzed is specified by epsilon. More specifically, the MIDI window of frames is created by including an epsilon number of frames before and after the initial MIDI measure boundary frame guess. For example, if epsilon is specified to be 3, then the MIDI window would be seven frames long, including the initial measure boundary guess in the middle. If epsilon is 0, then only the initial measure boundary guess would be compared to the XML measure boundary. Once the window is obtained, the Jaccard similarity is calculated individually between the XML frame, and each frame in the MIDI window of frames. Then, the comparison that has the lowest Jaccard distance ($1 - \text{Jaccard similarity}$) is stored. All these soft-similitude values across different pieces are then averaged to get a percentage of “similarity”. Thus, the soft-similitude metric provides a notion of approximate matches.

The strict-similitude metric is very similar to soft-similitude, however, there is one key difference. An XML and MIDI frame are only considered similar when their Jaccard distance is 0. In other words, both the XML and MIDI frames have to be identical to be considered a good MIDI measure boundary prediction, which is much more harsh of a metric. Thus, this metric provides a notion of exact matches.

Looking at Fig. 6, we can see that the normal binary representation consistently outperforms the unique binary and onset binary representations for all epsilon values for both the soft-similitude and strict-similitude metrics. This is significant because the unique binary representation is only a fraction of the size of the normal binary representation, and performs similarly while significantly reducing the computation time.

Additionally, both the soft and strict similitude metrics exhibit improvement as epsilon increases. However, this increase in similarity begins to taper off around an epsilon value of

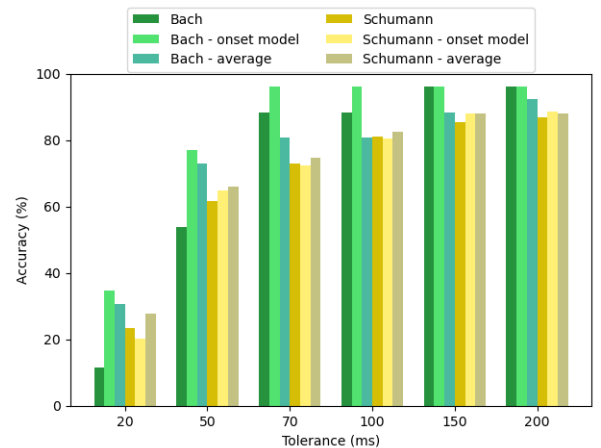


Fig. 5. Accuracy of Sheet Music Follower at different tolerances.

five. This plateau effect aligns with the natural variability of human musical performance, since it is hard for even the best musicians to achieve perfect timing on every note played.

From Fig. 7 we can see how the similitude metrics vary considerably among different pieces. This variation comes from multiple factors including the complexity of the piece and the stylistic choices of the performer. To illustrate, Bach's piece has higher similitude scores because the score is more simple with fewer large tempo variations. In contrast, there are more complex pieces, such as by Schumann or Mozart which receive lower scores on the similitude metrics.

IV. CONCLUSION

We have presented a system for inferring measure boundaries in MIDI performance files by aligning them with MusicXML score files using a modified DTW approach with three binary piano roll representations. Our evaluation shows that the system achieves high accuracy at relevant tolerance levels and performs consistently across different musical styles and performance conditions.

Some key insights emerged from this work. First, the choice of representation can considerably affect the accuracy of the alignment. In this case, our experiments show that the full binary representation generally outperforms onset and unique frame approaches. Second, the Jaccard distance metric was effective for comparing binary piano roll representations. Finally, our similitude metrics provide a solid evaluation framework that captures both exact and approximate matching performance.

Although our approach performs well on most pieces, we identified several cases in which the measure boundary prediction quality decreased. The measure boundary prediction quality tends to decrease when the performer takes more artistic liberties. We also noticed a slight decrease in quality when pieces are more complex and there are larger changes in tempo.

Ultimately, these results present a promising hybrid approach between all three binary representations. For exam-

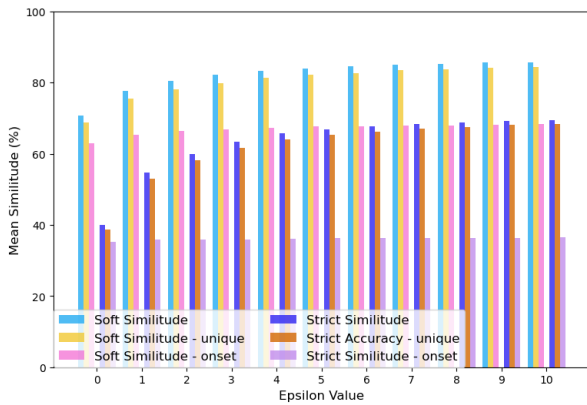


Fig. 6. Effect of epsilon values on soft-similitude and strict-similitude for normal binary roll, onset binary roll, and unique binary roll representations. These metrics are mean similitude (%) values for 12 songs in the asap dataset [6].

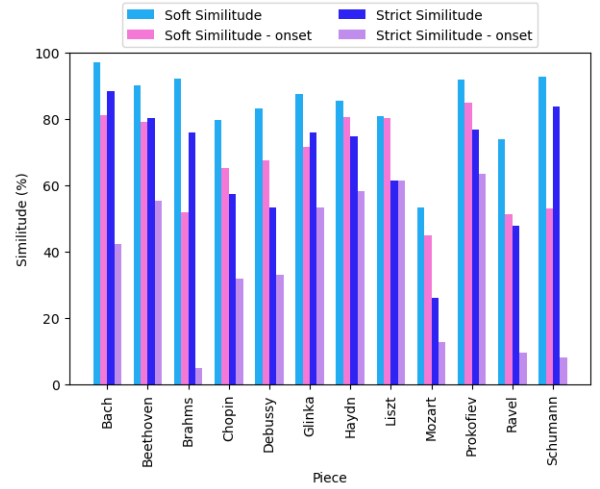


Fig. 7. Similitude of Sheet Music Follower for different pieces with an epsilon of 5.

ple, predicted measure boundaries for different binary roll representations could all be compared with score measure boundaries. Then, the boundary that aligns perfectly can be kept, or an average in time stamp of the inferred boundaries can be used to get the closest measure boundary.

The code for this project is available at [github], allowing for further research that builds on our contributions.

ACKNOWLEDGMENT

We would like to thank Professor TJ Tsai for his guidance and support throughout the project.

REFERENCES

- [1] "music21 Documentation — music21 Documentation." Accessed: May 08, 2025. [Online]. Available: <https://www.music21.org/music21docs/>
- [2] "pretty_midi — pretty_midi 0.2.10 documentation." Accessed: May 08, 2025. [Online]. Available: <https://craffel.github.io/pretty-midi/>
- [3] "C3S2_DTWvariants." Accessed: May 08, 2025. [Online]. Available: https://www.audiolabs-erlangen.de/resources/MIR/FMP/C3/C3S2_DTWvariants.html
- [4] S. Vikram et al., "Jaccard Similarity," Oracle Help Center. Accessed: May 08, 2025. [Online]. Available: <https://docs.oracle.com/en/database/oracle/oracle-database/23/vecse/jaccard-similarity.html>
- [5] M. Jadeja, "Jaccard Similarity Made Simple: A Beginner's Guide to Data Comparison," Medium. Accessed: May 08, 2025. [Online]. Available: <https://mayurdhvajsinhjadeja.medium.com/jaccard-similarity-34e2c15fb524>
- [6] F. Foscarin, fosfrancesco/asap-dataset. (Apr. 20, 2025). Jupyter Notebook. Accessed: May 08, 2025. [Online]. Available: <https://github.com/fosfrancesco/asap-dataset>